When you write a recursive function, you're always moving forward in the input. In this case, our list elements are iterated one by one, and the reduced value at the end is the biggest value in the list.

One great thing about Elixir is that you don't need to worry about having a recursive function that runs too long. Elixir is tail-call optimized, which means that it doesn't create a new stack entry for the final function call inside of a function. So you won't ever see a `StackOverflow` exception. Of course, you still need your base case or your function will run forever.

Wrapping Up

Pattern matching is one of the standout features of Elixir. You'll frequently use pattern matching, so it's important to get comfortable with it. The most basic form of pattern matching is the match operator =. Use the match operator to break values apart or for simple variable assignment. Pattern matching works with any term in Elixir, so you can use it for lists, maps, or any other data type.

Pattern matching is crucial in most of Elixir's control flow structures. `case` is the most common and foundational control flow structure. Use it to check a term against a number of candidate patterns. Use `with` to capture the successful outcome of a series of operations or to handle an error if something goes wrong. Finally, use `cond` when you need to check a statement's truthiness without the limitations of pattern matching.

Functions seem straightforward, but pattern matching supercharges them. You can overload a function not only with the number of arguments but also with pattern matching. Define the same function multiple times with different patterns in order to build recursive functions or functions that have different behavior for different input arguments.

Along with pattern matching, parallelism is often touted as Elixir's strength. The next chapter is all about processes, parallelism, and GenServers.